СРО№1




По дисциплине: Проектирование и применение NoSQL-технологий

Тема : Учет студентов и дисциплин

Вариант №2

Выполнил : Симиониди Анастас

Группа: ИС 24-22

Проверил: Рашидинов Д.Р



# Проблема и Цель

Проблема: В реляционных БД (SQL) для вывода транскрипта студента нужно делать 4–5 JOIN-запросов. Это медленно и ресурсоемко при частых чтениях.

Цель: Спроектировать и реализовать прототип NoSQL-подсистемы для быстрого учета успеваемости.

Задачи:

Спроектировать модель данных (Embed vs Ref).

Реализовать 10 Access Patterns (запросов).

Написать автотесты и скрипты заполнения.

# Требования (Функциональные и Нефункциональные)

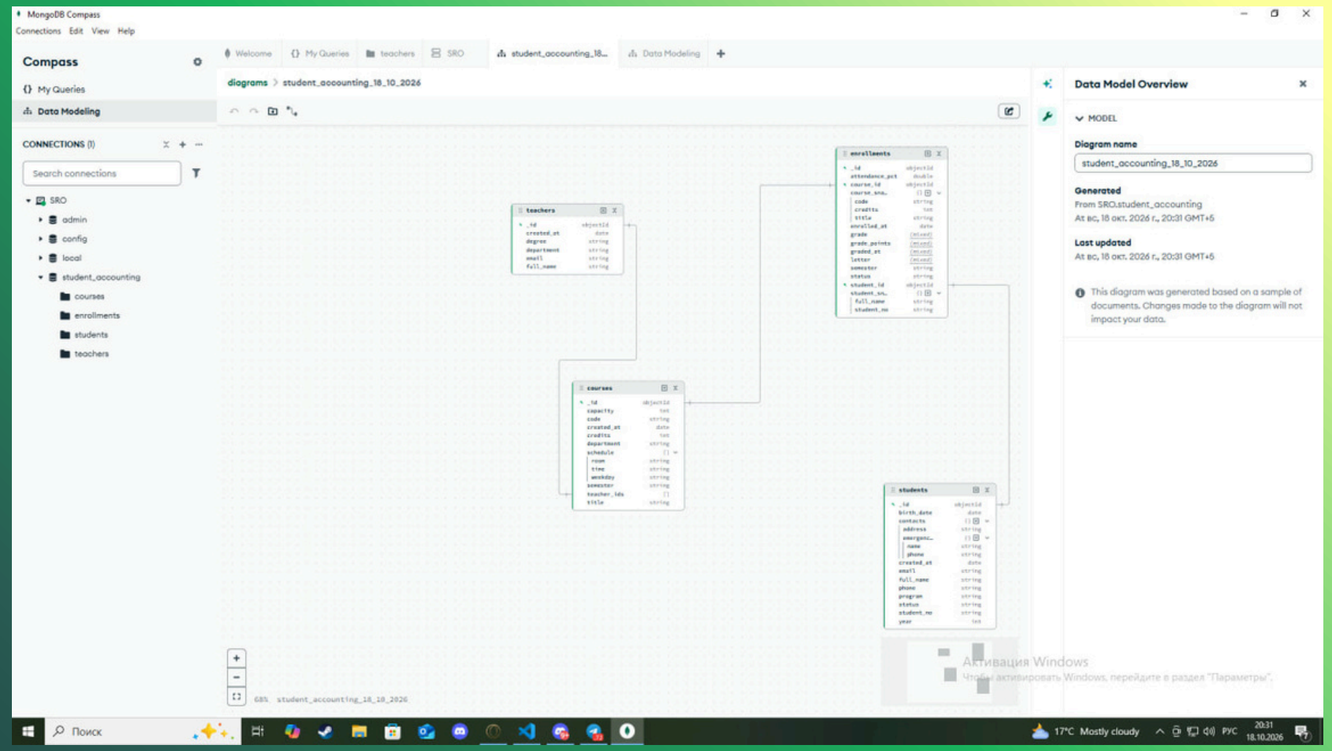
- **Функциональные (8):**
  - Карточки студентов, справочник курсов.
  - Запись на дисциплины и выставление оценок.
  - Формирование транскрипта.
  - Расчет GPA (взвешенный по кредитам).
  - Поиск должников (оценка < 60).
- **Нефункциональные (5):**
  - Скорость чтения до 50 мс (индексы).
  - Целостность (уникальные индексы).
  - **Историчность** (snapshot данных).
  - Воспроизводимость и безопасность.

# Выбор технологии (ADR)

- **PostgreSQL:** Мощный, но много JOIN и жесткая схема.
- **Neo4j:** Хорош для связей, но слаб в аналитике (GPA).
- **Cassandra:** Быстрая запись, но нет ad-hoc запросов.
- **MongoDB (Выбрана!):**
  - Гибкая схема (добавление полей без ALTER TABLE).
  - Aggregation Pipeline для расчета GPA.
  - Горизонтальное масштабирование (Sharding).

# Модель данных

- 4 коллекции: students, courses, teachers, enrollments.
- **Embed (Встраивание):** Контакты студента, расписание курса (читаются всегда вместе).
- **Reference (Связи):** Студент ↔ Курс (через ID).
- **Snapshot (Снимок):** В коллекции enrollments дублируются ФИО студента, название курса и кредиты на момент выставления оценки.



# Архитектура и индексы

- **Стек:** Python 3 → PyMongo → MongoDB 7.x.
- **Индексы — залог скорости:**
  - uq_student_no (Уникальный номер зачетки).
  - ix_enr_student_sem (Быстрый поиск транскрипта).
  - ix_enr_grade (Быстрый поиск должников).
- **Результат:** Запросы идут через IXSCAN (Index Scan), а не COLLSCAN (полный перебор).

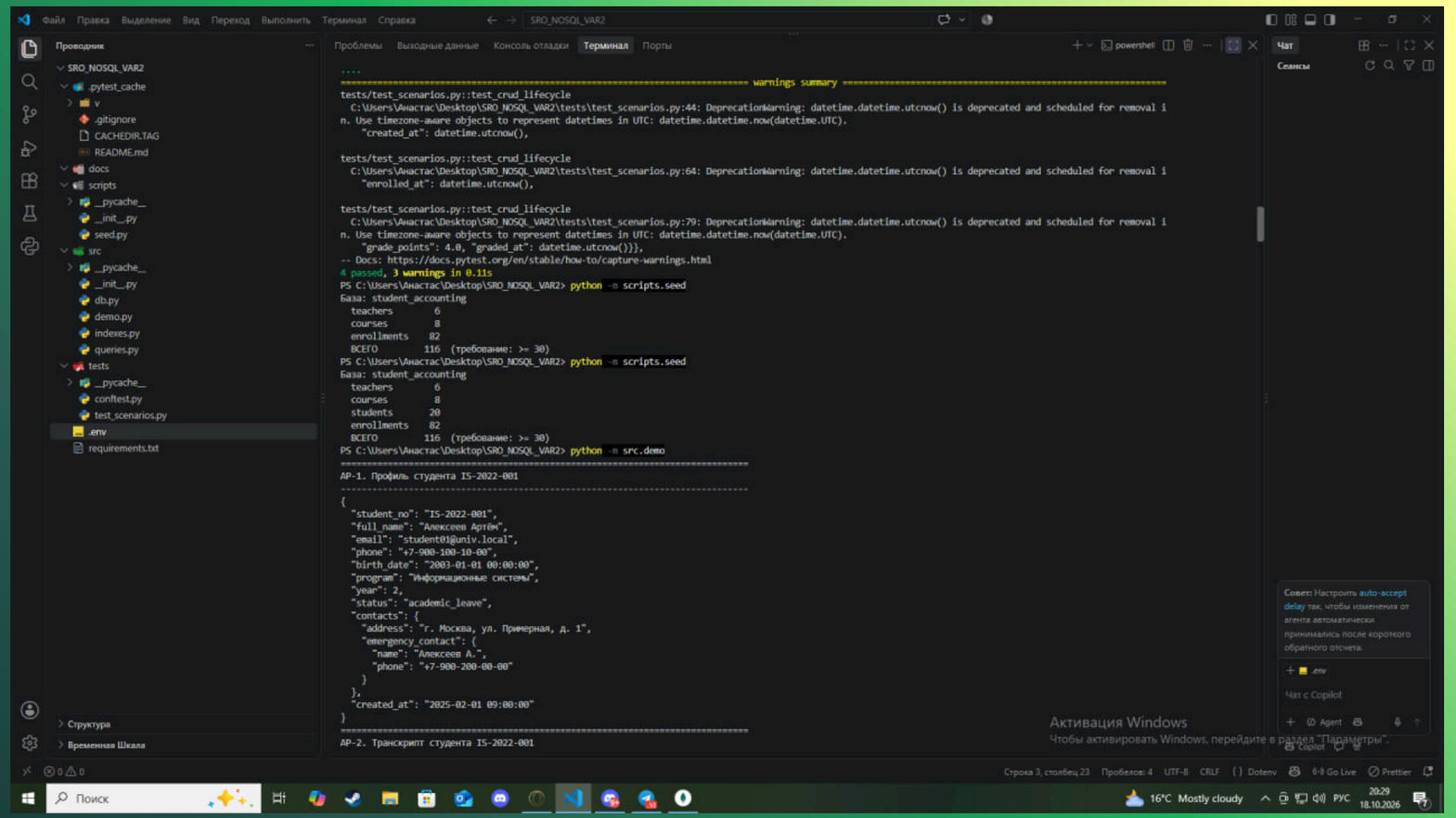
# Реализация (Код)

- Написан скрипт заполнения (114 синтетических документов).
- Реализовано 10 запросов.
- **Сложный запрос (АР-7): Расчет GPA.**
- Использование $group, $multiply и $divide для взвешивания оценки на количество кредитов.

```python
def students_gpa(db: Database, min_gpa: float = 0.0) -> list[dict]:
    """АР-7 (нетривиальный). GPA, взвешенный по кредитам, из snapshot-полей."""
    pipeline = [
        {"$match": {"grade": {"$ne": None}}},
        {"$group": {
            "_id": "$student_id",
            "student_no": {"$first": "$student_snapshot.student_no"},
            "full_name": {"$first": "$student_snapshot.full_name"},
            "courses": {"$sum": 1},
            "total_credits": {"$sum": "$course_snapshot.credits"},
            "weighted_points": {"$sum": {"$multiply": ["$grade_points", "$course_snapshot.credits"]}},
        }},
        {"$addFields": {
            "gpa": {"$round": [{"$divide": ["$weighted_points", "$total_credits"]}, 2]}
        }},
        {"$match": {"gpa": {"$gte": min_gpa}}},
        {"$sort": {"gpa": -1, "student_no": 1}},
        {"$project": {
            "_id": 0, "student_no": 1, "full_name": 1,
            "gpa": 1, "courses": 1, "total_credits": 1,
        }},
    ]
    return list(db.enrollments.aggregate(pipeline))
```

# Результаты запросов и тестирование

- Скрипт demo.py выводит работу всех 10 запросов (топ должников, статистика).
- Сравнение планов: Индекс = IXSCAN, Без индекса = COLLSCAN.
- **Тестирование:** 4 сценария на pytest (CRUD, расчет GPA, историчность, дубликаты).



# Ограничения и Выводы

- **Ограничения:** В тестовых данных 1 семестр.
  Нет авторизации (учебный прототип).
- **Выводы:**
  - MongoDB отлично подходит для быстрого чтения транскриптов.
  - Гибридная схема (Embed + Ref + Snapshot) решает проблему историчности данных.
  - Встроенная агрегация избавляет от сложных вычислений на стороне клиента.

Ссылка на GitHub

Спасибо за
внимание!

- Исходный код, тесты и отчет доступны в репозитории:
https://github.com/Anastas-Simionidi/No_sql.git

